

ULTIMATE EXAM GUIDE — Hesaplama Kuramı (BLM2502)

Объясняю как для осла. Ноль воды. Только то, что на экзамене.

Экзамен по книге Sipser, "Introduction to the Theory of Computation". Темы из реальных прошлых экзаменов (vize + final + örnek sorular): 1. DFA — строить автомат 2. NFA — строить + переводить в DFA 3. Regular Expressions (regex) 4. Pumping Lemma — доказать что язык НЕ регулярный 5. CFG — грамматика 6. CNF — Chomsky Normal Form 7. PDA — стековый автомат 8. Pumping Lemma для CFL + классификация языков 9. Turing Machines + Decidability (только финал)

Учи в этом порядке. Каждый раздел = шаблон решения + пример с прошлого экзамена.

0. БАЗА (выучить наизусть, это спрашивают True/False вопросами)

- **Язык (language)** = множество строк. Просто набор слов. ВСЁ.
- **Σ (sigma)** = алфавит = набор букв. Напр. $\Sigma = \{a, b\}$.
- **w** = строка (слово). **ϵ (эпсилон)** = пустая строка (длина 0).
- **Σ^*** = все возможные строки из этих букв, включая ϵ .
- **|w|** = длина строки.

Иерархия машин (от слабой к сильной):

```
DFA = NFA = Regex    → ловят REGULAR языки
PDA = CFG             → ловят CONTEXT-FREE языки (сильнее)
Turing Machine        → ловит всё что вообще вычислимо (самый сильный)
```

ФАКТЫ для True/False (реальные вопросы с vize 2022): - "Язык это множество строк" → **TRUE**. - Regular языки ЗАМКНУТЫ под: Union, Intersection, Complement, Concatenation, Star → **закрываются под ВСЕМИ**. - Каждый DFA это также NFA → **TRUE** (DFA это частный случай NFA). - NFA ловит языки, которые DFA не может → **FALSE** (они одинаковы по силе!). - Один автомат ловит ровно один язык, но один язык можно ловить разными автоматами → **TRUE**. - Если A нерегулярный, то A бесконечный → **TRUE** (все конечные языки регулярны). - $A \circ B$ regular → **TRUE** (конкатенация) context-free → **TRUE** (regular $\subset$ context-free, конкатенация regular даёт regular, а regular ВСЕГДА и context-free). - У regex есть PDA → **TRUE** (regex $\subset$ regular $\subset$ CF, а у CF есть PDA). - Pumping lemma доказывает что язык РЕГУЛЯРНЫЙ → **FALSE** (она доказывает что НЕ регулярный). - CFG не двусмысленна если есть >1 дерева разбора → **FALSE** (наоборот: >1 дерева = ДВУСМЫСЛЕННА). - В DFA из каждого состояния выходит ровно $|\Sigma|$ переходов → **TRUE**.

1. DFA — Deterministic Finite Automaton

Что это: машина с кружочками (состояния) и стрелками (переходы). Читает строку слева направо буква за буквой. Закончила в зелёном кружке (ассерт) → строку принимает.

Правила DFA (детерминированный): - Из КАЖДОГО состояния на КАЖДЮЮ букву ровно ОДНА стрелка. Не больше, не меньше. - НЕТ ϵ -переходов (пустых стрелок).

Обозначения: стартовое состояние — стрелка из ниоткуда (или треугольник). Ассер-состояние — двойной кружок.

АЛГОРИТМ "как строить DFA":

1. Спроси: "что я должен ЗАПОМНИТЬ, читая строку?" Это и есть состояния.
2. Нарисуй стартовое состояние.
3. Для каждого состояния проведи стрелку на каждую букву.
4. Помечай ассерт те состояния, где условие выполнено.
5. ПРОВЕРЬ на 2-3 строках (одна должна приниматься, одна — нет).

Частые шаблоны (ЗАЗУБРИТЬ):

"содержит подстроку X" (напр. 0101): делай цепочку состояний — каждое = "сколько символов подстроки я уже накопил". Дойдя до конца цепочки → ассерт-ловушка (там и остаёшься).

"заканчивается на X": то же, но ассерт = последнее состояние цепочки, и если ломается — откатываешься назад.

"чётное число нулей": 2 состояния — Even и Odd. На 0 переключаешься, на 1 стоишь. Start=Even=accept.

"длина делится на 3": 3 состояния по кругу $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0$. Accept= q_0 .

"X в третьей позиции с конца": запоминай последние 3 символа $\rightarrow$ 8 состояний (или сделай NFA, потом переведи).

ПРИМЕР (Osman örnek soru 1):

$L = \{w \mid w \neq ba \text{ И } w \text{ не содержит } bab\}$ над $\Sigma = \{a, b\}$.

Думай: надо (1) НЕ принять ровно строку "ba", (2) НЕ принять ничего с "bab". Строй так: отслеживай прогресс к "bab" (состояния: видел "b", видел "ba", видел "bab"=мёртвое). Отдельно пометь, что строка "ba" целиком запрещена.

```
start q0 --a--> q1 (акцепт)
q0 --b--> q2 (акцепт, видел "b")
q1 --a,b--> q1 ... (продолжаем, "bab" следим)
q2 --a--> q3 (видел "ba" — это запретная строка, НЕ акцепт пока)
q2 --b--> q2
q3 --b--> q4 (видел "bab" начало... осторожно)
q3 --a--> q1
qDEAD = поймали "bab" → ловушка, не акцепт
```

Ключ: состояние после "ba" — НЕ акцепт (т.к. ровно "ba" запрещена), но если дальше идёт буква и нет "bab" — снова акцепт. Поймал "bab" → мёртвая ловушка навсегда.

2. NFA — Nondeterministic Finite Automaton

Отличие от DFA: - Из состояния на одну букву может быть НЕСКОЛЬКО стрелок (или НИ ОДНОЙ). - Можно ϵ -переходы (прыжок без чтения буквы). - Принимает, если ХОТЬ ОДИН путь приводит в акцепт.

Зачем: NFA рисовать ЛЕГЧЕ. Особенно для "содержит X" или для regex.

АЛГОРИТМ "regex $\rightarrow$ NFA" (по кусочкам, теорема Клини):

- Буква a : $\bigcirc \xrightarrow{a} \odot$
- $R_1 R_2$ (конкатенация): соедини конец R_1 ϵ -стрелкой с началом R_2 .
- $R_1 \cup R_2$ (или): новый старт с ϵ -стрелками в оба.
- R^* (звезда): новый старт-акцепт, ϵ в R , конец R обратно ϵ в начало.

ПРИМЕР (Osman örnek soru 2):

NFA на 5 состояний для $a(cb)^* \cup c(ba)^*$:

```
      ε      a      c      b
start ----> q1 ---> q2 <----- (cb loop)
      \ε
      ---> q5 ---> ...c(ba)*
```

Конкретно:

```
q0 --a--> q1 (акцепт).  q1 --c--> q2.  q2 --b--> q1.  [ветка a(cb)*]
q0 --c--> q3 (акцепт).  q3 --b--> q4.  q4 --a--> q3.  [ветка c(ba)*]
```

5 состояний: q_0, q_1, q_2, q_3, q_4 . Акцепт: q_1 и q_3 . q_0 — стартовый, ловит обе ветки ($a...$ или $c...$).

АЛГОРИТМ "NFA $\rightarrow$ DFA" (Subset Construction / конструкция подмножеств) — ОЧЕНЬ ЧАСТО НА ЭКЗАМЕНЕ:

- Стартовое состояние DFA** = {старт NFA} + всё, куда можно дойти по ϵ (это называется ϵ -closure).
- Для этого множества и для КАЖДОЙ буквы: куда можно попасть из ЛЮБОГО состояния множества? Собери все эти состояния в новое множество (+ ϵ -closure).
- Каждое новое множество = новое состояние DFA. Повторяй пока не появляются новые множества.
- Акцепт-состояния DFA** = любое множество, содержащее хоть одно акцепт NFA.

5. Множество $\emptyset$ (пустое) = мёртвая ловушка.

ШАБЛОН ТАБЛИЦЫ (делай ВСЕГДА так):

Множество (DFA-состояние)	на 0	на 1
{q0} (старт)	...	...
{q0,q1}	...	...
...	...	...

Заполняй построчно. Новое множество встретил → добавь строку. Готово когда новых нет.

3. REGULAR EXPRESSIONS (regex)

Символы: - $\cup$ или $+$ = или. $*$ = ноль или больше повторов. Конкатенация = просто рядом. - Σ^* = вообще всё. ϵ = пустая строка.

АЛГОРИТМ "построить regex":

Раздели язык на части → склей через $\cup$ и конкатенацию. - "содержит X" → $\Sigma^* X \Sigma^*$ - "начинается с 1" → $1 \Sigma^*$ - "заканчивается на 0" → $\Sigma^* 0$ - "чётное число a" → $(b^* a b^* a b^*)^*$ (a идут парами) - "хотя бы три единицы" → $0^* 1 0^* 1 0^*$ (три обязательные 1, между ними что угодно)

ПРИМЕР (Osman örnek soru 3):

$L = \{w \mid \text{начинается с 1, заканчивается на 0, чётное число подстрок "01"}\}$ над $\Sigma = \{0,1\}$. Начинается 1, кончается 0. Подсчёт "01" чётный — это хитро. Ответ-скелет: $1 1^* (0 0^* 1 1^* 0 0^* 1 1^*)^* 0 0^*$ — но точную форму подгоняй под "чётное число 01". Метод: считай блоки. Каждый переход $0 \rightarrow 1$ это одно "01". Чётное число таких переходов.

4. PUMPING LEMMA (для REGULAR) — доказать что язык НЕ регулярный

ИДЕЯ как для осла: если язык регулярный, у него конечная память (p состояний). Возьми ДЛИННУЮ строку — машина обязана заикнуться (войти в кружок дважды). Этот кусок-цикл можно "качать" (повторять). Если после качания строка ВЫПАДАЕТ из языка → противоречие → язык НЕ регулярный.

АЛГОРИТМ (зазубрить эти 5 шагов, всегда одинаково):

1. "Предположим, L регулярный." (от противного). Тогда есть pumping length p.
2. **Выбери хитрую строку** $s \in L$, где $|s| \geq p$. (Это самый важный шаг — выбирай строку с равными счётчиками, напр. $a^p b^p$.)
3. **По лемме** $s = xyz$, причём: - $|y| > 0$ (y непустой), - $|xy| \leq p$ (x и y в первых p символах), - $xy^iz \in L$ для всех $i \geq 0$.
4. **Раз** $|xy| \leq p$, то x и y состоят ТОЛЬКО из первых символов (напр. только из a). Значит $y = a^k$, $k > 0$.
5. **Качай** $i=2$ (или $i=0$): получаем $a^{p+k} b^p$. Теперь a и b НЕ равны → строка $\notin L$. **Противоречие!** $\Rightarrow L$ НЕ регулярный. ■

ПРИМЕР классики: $L = \{0^n 1^n \mid n \geq 0\}$ НЕ регулярный.

- $s = 0^p 1^p$.
- xy в первых p → y = только нули = 0^k , $k > 0$.
- $xy^2z = 0^{p+k} 1^p$. Нулей больше чем единиц → $\notin L$. Противоречие. ■

ПРИМЕР (Osman örnek soru 4): $L = \{www \mid w \text{ начинается с 0}\}$.

- Выбери $s = (0^p)(0^p)(0^p)$? Нет — нужно где видно тройное повторение.
- Бери $s = 0^p 1 0^p 1 0^p 1$ (это www где $w = 0^p 1$). $|s| \geq p$.
- $|xy| \leq p \Rightarrow y$ только нули из ПЕРВОГО блока. Качаем → первый блок нулей растёт, остальные нет → уже не www (три части не равны) → $\notin L$. Противоречие. ■

ГЛАВНАЯ ОШИБКА студентов: даёшь машине выбирать у. НЕТ. Машина выбирает р и разбиение хуз, Ты выбираешь s и i. Лемма "за тебя" только в шаге выбора строки.

5. CFG — Context-Free Grammar (контекстно-свободная грамматика)

Что это: правила переписывания. Большие буквы = переменные (variables), маленькие = терминалы (буквы языка). S = старт.

Пример: $S \rightarrow aSb \mid \epsilon$ генерирует $\{a^n b^n\}$.

АЛГОРИТМ "построить CFG":

- **Равные счётчики $a^n b^n$:** $S \rightarrow aSb \mid \epsilon$ (добавляй a слева, b справа одновременно).
- **Палиндромы:** $S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$.
- **$a^n b^m$, $n=m$:** см. выше. $n \neq m$ или $n \leq m$ — добавь "лишние" в нужную сторону.
- **$a^n b^m c^n$** (две независимые пары): $S \rightarrow XY$, $X \rightarrow aXb \mid \epsilon$, $Y \rightarrow cY \mid \epsilon$.

ПРИМЕР (Osman örnek soru 6): $L = \{ab^na^a \mid n \geq 0\} = a$, потом n штук b, потом n штук a, потом a.

Думай: между внешними a...a сидит b^na^n .

```
S → a B a
B → b B a | ε
```

Проверка: $n=0 \rightarrow "aa" \checkmark$. $n=1 \rightarrow a(baa) \rightarrow "abaaa" \dots$ подгони под точное условие ab^na^a .

ПРИМЕР (Osman örnek soru 9): $B = \{a^i b^j \mid i \leq j \leq 2i\}$, нужна ОДНОЗНАЧНАЯ (unambiguous) CFG.

Каждое a даёт 1 или 2 b. Значит:

```
S → a S b | a S b b | ε
```

Каждый шаг: одно a и (одно ИЛИ два) b. Так j между i и 2i. $\checkmark$

6. CNF — Chomsky Normal Form

Что это: грамматика, где КАЖДОЕ правило одного из двух видов: - $A \rightarrow BC$ (ровно две переменные справа), или - $A \rightarrow a$ (ровно один терминал). - Исключение: $S \rightarrow \epsilon$ разрешено только для старта.

АЛГОРИТМ перевода в CNF (делай В ЭТОМ ПОРЯДКЕ):

1. **START:** добавь новый старт $S_0 \rightarrow S$ (чтобы старый старт не был в правых частях).
2. **DEL ϵ :** убери ϵ -правила (кроме S_0). Для каждого $A \rightarrow \epsilon$: везде где A встречается справа, добавь версию без A. Потом удали $A \rightarrow \epsilon$.
3. **UNIT:** убери единичные правила $A \rightarrow B$. Замени: что генерил B — теперь генерит и A.
4. **TERM:** в длинных правилах замени терминалы на новые переменные: $A \rightarrow aB \rightarrow A \rightarrow Ua B$, $Ua \rightarrow a$.
5. **BIN:** разбей длинные правила (>2 символов) на пары: $A \rightarrow BCD \rightarrow A \rightarrow B X$, $X \rightarrow CD$.

ПРИМЕР (Osman örnek soru 7): перевести в CNF:

```
S → ASA | aB
A → B | S
B → b | ε
```

Шаг 1 (START): $S_0 \rightarrow S$. **Шаг 2 (DEL ϵ):** $B \rightarrow \epsilon$. Убираем. Где B справа: $A \rightarrow B$ и $S \rightarrow aB$. - $S \rightarrow aB$ даёт ещё $S \rightarrow a$. - $A \rightarrow B$ — B мог быть ϵ , значит $A \rightarrow \epsilon$ появляется. Записываем $A \rightarrow \epsilon$ временно, потом тоже чистим. - После чистки $A \rightarrow \epsilon$: в $S \rightarrow ASA$ A может

выпасть $\rightarrow$ добавь $S \rightarrow SA, S \rightarrow AS, S \rightarrow S$. Удаляем все $\rightarrow \epsilon$. **Шаг 3 (UNIT):** убрать $S \rightarrow S$ (выкинуть), $A \rightarrow S, A \rightarrow B, S \rightarrow S$. - $A \rightarrow B: B \rightarrow b$, значит $A \rightarrow b$. - $A \rightarrow S: A$ получает все правила S . - $S \rightarrow S: S$ получает все правила S . **Шаг 4 (TERM):** $S \rightarrow aB \rightarrow$ введи $U \rightarrow a, S \rightarrow UB$. **Шаг 5 (BIN):** $S \rightarrow ASA \rightarrow S \rightarrow A X1, X1 \rightarrow SA$. Финал — все правила вида $A \rightarrow BC$ или $A \rightarrow a$. Не забудь выписать 4-tuple (V, Σ, R, S_0) .

7. PDA — Pushdown Automaton (стековый автомат)

Что это: NFA + СТЕК (стопка тарелок). Может push (положить) и pop (снять) символ сверху стека. Стек = память для счёта. Поэтому PDA умеет $\{a^n b^n\}$ (DFA не умеет).

Запись перехода: $\delta(q, a, X) = (p, Y)$ = "в состоянии q , читаю букву a , СВЕРХУ стека $X \rightarrow$ иди в p , замени X на Y ". - a может быть ϵ (не читать букву). - $\$$ = маркер дна стека (кладут первым).

АЛГОРИТМ "построить PDA" для $a^n b^n$ -типа:

1. Положи $\$$ на дно (стартовый ϵ -переход).
2. **Фаза push:** читаешь $a \rightarrow$ push A на стек (по разу за каждую a).
3. **Фаза pop:** читаешь $b \rightarrow$ pop A (снимаешь по A за каждую b).
4. Если в конце на стеке только $\$$ $\rightarrow$ акцепт (стек опустел = a и b сошлись).

ПРИМЕР (Osman örnek soru 10): $L = \{a^n b^m \mid n, m \geq 1, n < m \leq 2n\}$.

Каждая a даёт 1 или 2 b . Стек считает.

```
q0 --ε, ε→$--> q1          (положить дно)
q1 --a, ε→A--> q1          (на каждую a положить A)
q1 --b, A→ε--> q2          (первая b начинает снимать)
q2 --b, A→ε--> q2          (b снимает A — обычный счёт)
q2 --b, A→A--> q2          ИЛИ дай каждой A покрыть 2 b (для n < m ≤ 2n)
q2 --ε, $→ε--> qaccept
```

Идея для $n < m \leq 2n$: положи по 2 маркера за каждую a (даёт верх $2n$), b снимают, требуй чтобы b было строго больше n но $\leq 2n$. На экзамене распиши недетерминированно: PDA "угадывает" сколько b покрывает каждая a .

АЛГОРИТМ "CFG $\rightarrow$ PDA" (механически, всегда работает):

1. Три состояния: $q_{start} \rightarrow q_{loop} \rightarrow q_{accept}$.
2. q_{start} : push S , иди в q_{loop} .
3. В q_{loop} , на КАЖДОЕ правило $A \rightarrow w: \delta(q_{loop}, \epsilon, A) = (q_{loop}, w)$ (замени переменную на правую часть).
4. На каждый терминал $a: \delta(q_{loop}, a, a) = (q_{loop}, \epsilon)$ (читаешь = снимаешь со стека).
5. $\delta(q_{loop}, \epsilon, \$) = (q_{accept}, \epsilon)$.

8. КЛАССИФИКАЦИЯ ЯЗЫКОВ + Pumping Lemma для CFL

Вопрос на финале (homework q1, q7): "регулярный / контекстно-свободный (но не регулярный) / ни то ни сё?"

ДЕРЕВО РЕШЕНИЙ:

1. Можешь сделать DFA/regex? $\rightarrow$ **REGULAR**. (счёт конечный, или просто паттерн)
2. Не regular, но можешь сделать CFG/PDA (ОДИН счётчик/стек)? $\rightarrow$ **CONTEXT-FREE**. - Признак CF: одно сравнение/вложенность. $a^n b^n$, палиндромы, $a^n b^n c^m$.
3. Нужно ДВА независимых счёта одновременно? $\rightarrow$ **НЕ context-free** (ни то ни сё). - Признак: $a^n b^n c^n$ (три равных), ww (копия), $a^n b^n c^n$ — стека не хватает.

Быстрые ориентиры: - $a^n b^n c^n \rightarrow$ НЕ CF (нужно считать три вещи, стек один). - ww (w повтор) $\rightarrow$ НЕ CF. Но ww^R (w потом ОБРАЩЁННЫЙ w) $\rightarrow$ CF (палиндром, стек справляется). - $a^n b^n \rightarrow$ CF не regular. - $\{a^i b^j : i \leq j \leq 2i\} \rightarrow$ CF.

PUMPING LEMMA для CFL (доказать что язык НЕ контекстно-свободный):

Та же логика, но строка делится на **5 частей**: $s = uvxyz$. Условия: $|vy| > 0$, $|vxy| \leq p$, и $u v^i x y^i z \in L$ для всех i . **Качаешь v и y ОДНОВРЕМЕННО**. Покажи что строка выпадает $\rightarrow$ не CF.

ПРИМЕР: $L = \{a^n b^n c^n\}$ не CF.

- $s = a^p b^q c^r$. $|vxy| \leq p \Rightarrow vxy$ влезает максимум в ДВА типа букв (не во все три).
- Качаем $i=2$: один или два символа растут, третий нет $\rightarrow$ счётчики разошлись $\rightarrow \notin L$. ■

9. TURING MACHINES + DECIDABILITY (только ФИНАЛ)

Turing Machine (TM): лента бесконечная + головка читает/пишет/двигается влево-вправо. Самая мощная машина. Может ВСЁ что алгоритмически вычислимо.

Переход TM: $\delta(q, a) = (p, b, R)$ = "в q читаю $a \rightarrow$ пиши b , иди в p , двинь головку Right (или L)".

Виды: - **Decidable (разрешимый)** = TM ВСЕГДА останавливается (да/нет). - **Turing-recognizable (распознаваемый)** = TM останавливается на "да", но на "нет" может крутиться вечно.

ГЛАВНЫЕ ФАКТЫ (спрашивают): - A_{DFA} (принимает ли DFA строку?) $\rightarrow$ **DECIDABLE**. - A_{CFG} (генерирует ли CFG строку?) $\rightarrow$ **DECIDABLE**. - A_{TM} (принимает ли TM строку?) $\rightarrow$ **распознаваемый, но НЕ decidable** (Halting Problem!). - **Halting Problem** (остановится ли TM?) $\rightarrow$ **НЕ decidable** (доказано диагональю). - Иерархия: $regular \subset context-free \subset decidable \subset recognizable$.

КАК СТРОИТЬ TM (для $a^n b^n c^n$ — классика):

1. Прочитай a , сотри (поставь X), иди вправо.
2. Найди первую b , сотри (Y), иди вправо.
3. Найди первую c , сотри (Z), вернись в начало.
4. Повторяй. Если всё стёрлось ровно $\rightarrow$ accept. Если осталось лишнее $\rightarrow$ reject.

ШПАРГАЛКА "ЧТО ДЕЛАТЬ КОГДА ВИЖУ ЗАДАЧУ"

Виджу в задаче	Делаю
"construct DFA"	таблица состояний = что помню; шаблоны из §1
"5-state NFA for regex"	разбей regex по §2, склей кусочки
"convert NFA to DFA"	subset construction, таблица §2
"regular expression for..."	разбей на части, $\cup$ и конкатенация §3
"prove not regular"	Pumping Lemma 5 шагов §4
"CFG for language"	шаблоны §5 ($a^n b^n \rightarrow S \rightarrow aSb \epsilon$)
"convert to CNF"	5 шагов START-DEL-UNIT-TERM-BIN §6
"construct PDA"	стек считает; §7
"CFG to PDA"	3 состояния, механика §7
"regular/CF/neither"	дерево решений §8
"prove not context-free"	Pumping Lemma 5 частей §8
"is it decidable"	факты §9

ТОП-5 ОШИБОК НА ЭКЗАМЕНЕ (НЕ ДЕЛАЙ):

1. **Pumping lemma:** сам выбираешь u — НЕТ. Ты выбираешь s и i , машина выбирает p и h .
2. **DFA:** забыл переход из состояния на какую-то букву — автомат СЛОМАН. Из каждого по $|\Sigma|$ стрелок.
3. **CNF:** забыл новый старт S_0 или порядок шагов — мусор. Порядок: $START \rightarrow DEL \rightarrow UNIT \rightarrow TERM \rightarrow BIN$.
4. **NFA $\rightarrow$ DFA:** забыл ϵ -closure или пустое множество $\emptyset$ как ловушку.
5. **Классификация:** перепутал ww (НЕ CF) и ww^R (CF). Reverse = CF, copy = НЕ CF.

Удачи. Прочитал? Пиши "TEST ME" — устрою жёсткий допрос по реальным вопросам.